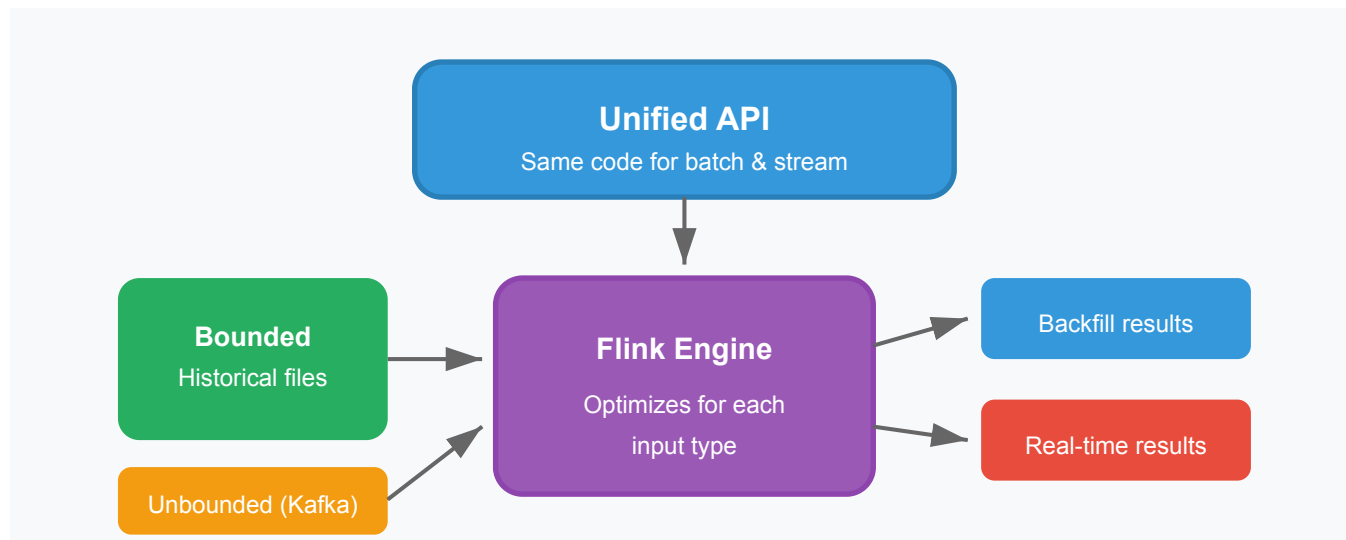


# Unified Batch and Stream Processing: Flink, Spark, Beam

 [systemdesignschool.io/fundamentals/batch-stream-unification](https://systemdesignschool.io/fundamentals/batch-stream-unification)



## Unified Processing

Lambda and Kappa architectures treat batch and stream as separate concerns—run them separately (Lambda) or use stream processing for everything (Kappa). Modern frameworks take a different approach: **treat batch and stream as the same computation over different data shapes**.

Write your pipeline once. Point it at historical files for a backfill. Point it at a live event stream for real-time results. Same code, same semantics, different input sources.

## The Unified Model

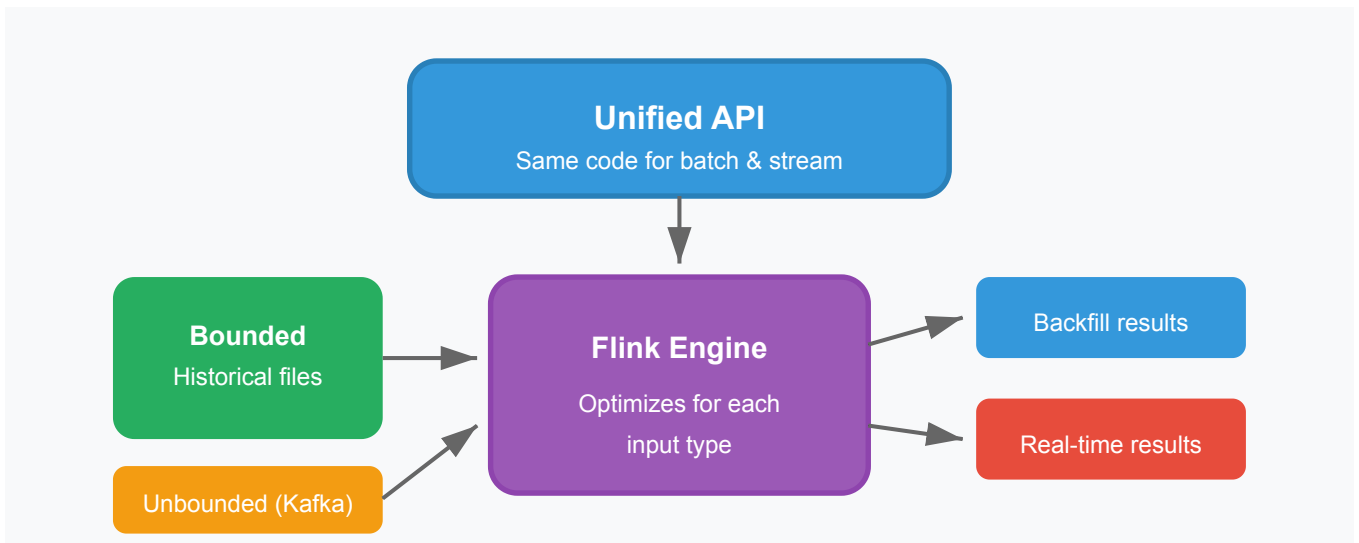
The distinction: batch processing operates on **bounded** data (finite, with a defined end), while stream processing operates on **unbounded** data (continuous, with no end). But the transformations—filter, map, aggregate, join—are conceptually identical.

A unified framework provides a single API for both. The execution engine handles the differences:

- For bounded data: optimize for throughput, read all data, produce complete results
- For unbounded data: optimize for latency, process continuously, handle late data

## Apache Flink: Stream-First Unification

Flink treats batch as a special case of streaming—a bounded stream. The same `DataStream` or `Table` API works for both.



Flink's execution engine adapts based on input:

- **Streaming mode:** Processes events as they arrive, maintains state continuously, handles watermarks and late data
- **Batch mode (BATCH execution):** Reads complete input, enables additional optimizations like sort-based joins, produces final results

### Strengths:

- Native event-time processing with watermarks built in
- Exactly-once processing via distributed snapshots (checkpoints)
- Sophisticated state management that survives failures
- True low-latency streaming (sub-second), not micro-batching

Flink excels at complex stateful applications where event-time correctness matters—session analysis, complex event processing, real-time machine learning features.

## Apache Spark: Batch-First Unification

Spark started as a batch engine and added streaming capabilities through Structured Streaming. The same DataFrame/Dataset API works for both modes.

```
# Same transformation code for batch or stream
df.groupBy("category").agg(sum("amount"))
```

The difference is in how you create the DataFrame:

- **Batch:** `spark.read.parquet("s3://data/orders/")`
- **Stream:** `spark.readStream.format("kafka").load()`

**How Structured Streaming works:** Rather than processing event-by-event, Spark collects events into micro-batches (typically every few hundred milliseconds to seconds) and processes each batch using the same engine as batch jobs. This provides streaming semantics while leveraging Spark's mature batch execution.

## Trade-offs:

- Micro-batching adds latency—typically seconds, not milliseconds
- True event-time semantics require explicit watermark configuration
- But teams already using Spark for batch can add streaming with minimal new learning

Spark is a good choice when you have significant Spark batch investments, need ML integration (MLlib), or when second-level latency is acceptable.

## Apache Beam: Portability Layer

---

Beam provides a unified programming model that runs on multiple execution engines: Flink, Spark, Google Cloud Dataflow, and others.

Write your pipeline in the Beam SDK. Deploy to whichever runner makes sense for your environment. The same pipeline definition can run on Dataflow in GCP, Flink on-premise, or Spark on EMR.

**The appeal:** Avoid vendor lock-in. Change execution engines without rewriting pipelines. Teams can standardize on Beam while using different infrastructure across environments.

**The trade-off:** Abstraction costs. Advanced features of specific engines (Flink's sophisticated state backends, Spark's ML integration) may not be fully exposed through Beam's unified API. You optimize for portability rather than maximizing any single engine's capabilities.

## Choosing a Framework

---

| Framework | Choose When                                                                                          |
|-----------|------------------------------------------------------------------------------------------------------|
| Flink     | You need sub-second latency, complex event-time processing, or sophisticated stateful operations     |
| Spark     | You have existing Spark batch pipelines, need ML integration, or second-level latency is acceptable  |
| Beam      | Multi-cloud portability matters, or you want to standardize on one API across diverse infrastructure |

For greenfield projects requiring low-latency streaming, Flink is often the strongest choice. For organizations with heavy Spark investments adding streaming capabilities, Spark Structured Streaming provides the gentlest migration path. For multi-cloud enterprises, Beam offers valuable flexibility.

## The Practical Impact

---

Unified processing simplifies development and operations:

- **One codebase** for batch and stream logic—no divergence between implementations

- **One test suite** that validates both modes
- **Easier backfills**—run the same code over historical data to rebuild outputs
- **Simpler reasoning**—same semantics whether data is bounded or unbounded

The batch-stream distinction becomes an input configuration rather than an architectural decision. You choose based on latency requirements and data availability, not because you need different code.

## What's Next

---

We've covered the core processing models: batch (MapReduce, Spark), stream (Flink, Kafka Streams), hybrid architectures (Lambda, Kappa), and unified frameworks. The following articles address practical patterns: ETL vs ELT, backfill strategies, error handling with dead-letter queues, and real-time analytics architectures.